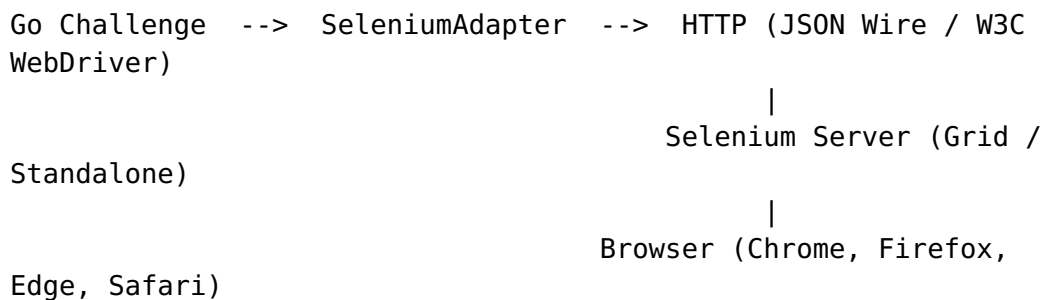


selenium-adapter

Selenium Adapter

The Selenium adapter implements BrowserAdapter using the W3C WebDriver protocol. It communicates with a Selenium Grid, standalone server, or any W3C-compliant WebDriver implementation (Selenoid, Moon, Aerokube) over HTTP.

Architecture



Each adapter method translates to one or more WebDriver HTTP calls. The adapter creates a session during Initialize, executes commands against that session, and deletes the session on Close. All communication uses JSON over HTTP -- no binary protocols or WebSocket connections are needed.

W3C WebDriver Protocol

The adapter uses the W3C WebDriver standard (successor to the JSON Wire Protocol). Key protocol details:

- **Session creation:** POST /session with capabilities.alwaysMatch
- **Element location:** POST /session/{id}/element with using: "css selector"
- **Element ID format:** W3C uses the well-known key element-6066-11e4-a52e-4f735466cecf; the adapter also supports the legacy ELEMENT key
- **JavaScript execution:** POST /session/{id}/execute/sync
- **Screenshots:** GET /session/{id}/screenshot returns base64-encoded PNG

- **Session deletion:** DELETE /session/{id}

Prerequisites

1. A running Selenium server (Grid or standalone) accessible over HTTP
2. The target browser installed on the machine running the Selenium server
3. The corresponding browser driver (ChromeDriver, GeckoDriver, etc.)

Configuration

BrowserConfig

The Initialize method accepts the standard BrowserConfig:

```
type BrowserConfig struct {  
    BrowserType string `json:"browser_type" // "chrome",  
        "firefox", "edge", "safari"  
    Headless    bool   `json:"headless"`  
    WindowSize [2]int  `json:"window_size" // [width, height]  
    ExtraArgs   []string `json:"extra_args"`  
}
```

Browser type mapping:

BrowserConfig Value	WebDriver browserName	Options Key
chrome, chromium	chrome	goog:chromeOptions
firefox, gecko	firefox	moz:firefoxOptions
edge, msedge	MicrosoftEdge	ms:edgeOptions
safari	safari	(no options key)

When Headless is true, --headless is added to the browser arguments. Window size is passed as --window-size=W,H. Any strings in ExtraArgs are appended to the arguments list.

Constructor

```
adapter := userflow.NewSeleniumAdapter("http://localhost:4444")
```

The single argument is the base URL of the Selenium WebDriver server. Trailing slashes are stripped. The adapter creates an internal http.Client with a 30-second timeout.

API Reference

Initialize

```
err := adapter.Initialize(ctx, userflow.BrowserConfig{
    BrowserType: "chrome",
    Headless:    true,
    WindowSize: [2]int{1920, 1080},
})
```

Creates a new WebDriver session. The session ID is stored internally and used for all subsequent operations.

Navigate, Click, Fill, SelectOption

```
err := adapter.Navigate(ctx, "https://example.com")
err = adapter.Click(ctx, "#submit-button")
err = adapter.Fill(ctx, "#email", "user@example.com")
err = adapter.SelectOption(ctx, "#country", "US")
```

SelectOption uses JavaScript execution (document.querySelector + dispatchEvent) since WebDriver does not have a native select endpoint for arbitrary dropdowns.

IsVisible, WaitForSelector

```
visible, err := adapter.IsVisible(ctx, ".alert-success")
```

```
err = adapter.WaitForSelector(ctx, ".dashboard", 10*time.Second)
```

WaitForSelector polls every 200ms until the element is found or the timeout expires.

GetText, GetAttribute

```
text, err := adapter.GetText(ctx, "h1.title")
href, err := adapter.GetAttribute(ctx, "a.link", "href")
```

Screenshot, EvaluateJS

```
png, err := adapter.Screenshot(ctx)
result, err := adapter.EvaluateJS(ctx, "return document.title")
```

Screenshot returns base64-decoded PNG bytes. EvaluateJS uses the synchronous script execution endpoint.

NetworkIntercept

```
err := adapter.NetworkIntercept(ctx, "**/api/**", handler)
// Returns nil; handler will NOT fire.
```

Network interception is not supported by the WebDriver protocol. The method returns nil to avoid breaking challenge flows. Use a proxy-based solution (e.g., BrowserMob Proxy) for network interception with Selenium.

Close, Available

```
err := adapter.Close(ctx)
ok := adapter.Available(ctx) // Checks GET /status
```

Docker Setup for Selenium Grid

```
version: "3.8"
services:
  selenium-hub:
    image: selenium/hub:4.18
    ports:
      - "4444:4444"
  chrome:
    image: selenium/node-chrome:4.18
    depends_on:
      - selenium-hub
    environment:
      - SE_EVENT_BUS_HOST=selenium-hub
      - SE_EVENT_BUS_PUBLISH_PORT=4442
      - SE_EVENT_BUS_SUBSCRIBE_PORT=4443
  firefox:
    image: selenium/node-firefox:4.18
    depends_on:
      - selenium-hub
    environment:
      - SE_EVENT_BUS_HOST=selenium-hub
      - SE_EVENT_BUS_PUBLISH_PORT=4442
      - SE_EVENT_BUS_SUBSCRIBE_PORT=4443
```

Standalone mode (single browser, simpler setup):

```
services:
  selenium:
    image: selenium/standalone-chrome:4.18
    ports:
      - "4444:4444"
    shm_size: "2g"
```

Limitations vs Playwright

Feature	Selenium	Playwright
Network interception	Not supported (use proxy)	Native support
Multi-tab control	Limited	Full support
Protocol	HTTP (higher latency)	CDP WebSocket (lower latency)
Browser install	Manual driver management	Auto-downloads browsers
Cross-browser	Chrome, Firefox, Edge, Safari	Chromium, Firefox, WebKit
Speed	Slower (HTTP round-trips)	Faster (persistent connection)
Mobile emulation	Requires Appium	Built-in device emulation

Integration with Challenge Templates

```
adapter := userflow.NewSeleniumAdapter("http://selenium:4444")

challenge := userflow.NewBrowserFlowChallenge(
    "CH-SELENIUM-001",
    "Selenium Login Flow",
    "Verify login using Selenium WebDriver",
    nil,
    adapter,
    userflow.BrowserFlow{
        Name:      "selenium-login",
        StartURL:  "http://app:3000/login",
        Config: userflow.BrowserConfig{
            BrowserType: "chrome",
            Headless:    true,
            WindowSize:  [2]int{1920, 1080},
        },
        Steps: []userflow.BrowserStep{
            {Name: "enter-email", Action: "fill", Selector: "#email", Value: "test@test.com"},
            {Name: "enter-pass", Action: "fill", Selector: "#password", Value: "secret"},
            {Name: "submit", Action: "click", Selector: "button[type=submit]"},
            {Name: "verify", Action: "assert_visible", Selector: ".dashboard"},
        },
    },
)
```

```
    },  
)
```

Source Files

- Interface: pkg/userflow/adapter_browser.go
- Implementation: pkg/userflow/selenium_adapter.go